

Hashing

What is Hashing

Hashing maps data of arbitrary size to a fixed-size value (a hash) using a hash function, which is then used as an index into an array-like structure (a hash table) for near-instant lookup, insertion, and deletion.

Hash Table / Hash Map

A hash table stores key-value pairs. The hash function converts a key into an array index. In Python, this structure is built in as the dict type. Average-case time complexity for lookup, insert, and delete is $O(1)$ — this is what makes hashing so powerful for interview problems.

Collisions

Two different keys can hash to the same index — this is a collision. The two main resolution strategies are: Chaining (each array slot holds a linked list of all entries that hash there) and Open Addressing (probe for the next free slot). Python's dict uses open addressing internally.

Time Complexity

- Average case — Insert, Delete, Search: $O(1)$.
- Worst case (many collisions): $O(n)$, though modern implementations make this extremely rare in practice.
- Space: $O(n)$.

Hash Map Usage in Python

```
# dict is Python's built-in hash map
```

```
freq = {}
```

```
for char in "hello":
```

```
    freq[char] = freq.get(char, 0) + 1
```

```
print(freq) # {'h':1, 'e':1, 'l':2, 'o':1}
```

```
# set is a hash-based structure for uniqueness/fast lookup
```

```
seen = set()
```

```
seen.add(5)
```

```
print(5 in seen) #  $O(1)$  lookup -> True
```

```
# Two Sum using a hash map -  $O(n)$  instead of  $O(n^2)$ 
```

```
def two_sum(nums, target):  
  
    seen = {}  
  
    for i, num in enumerate(nums):  
  
        complement = target - num  
  
        if complement in seen:  
  
            return [seen[complement], i]  
  
        seen[num] = i
```

Why Hashing Matters for Interviews

A huge portion of 'optimize this' interview questions are solved by trading $O(n^2)$ brute force for $O(n)$ using a hash map to remember what you've already seen. If you see 'find pairs', 'count frequency', 'check duplicates', or 'group by', a hash map is almost always part of the optimal solution.

Common Interview Problems

- Two Sum, Three Sum using hash maps.
- Group Anagrams — group strings by sorted-character key.
- Longest Consecutive Sequence in an unsorted array, $O(n)$ using a set.
- First non-repeating character in a string.
- Subarray Sum Equals K using prefix sums + hash map.
- Check if two strings are anagrams of each other.